

Prévision de la consommation électrique avec un Stacked LSTM multivarié

Rapport du projet de fin de module

Par : Abdelhamid SAIDI
Année universitaire 2025–2026

Fichier source principal : LSTM-I.ipynb
module : Deep Learning

Table des matières

1	Contexte et objectif	3
2	Données utilisées	3
2.1	Source et nature du jeu de données	3
2.2	Variables d'entrée	3
3	Prétraitement et ingénierie des variables	3
3.1	Nettoyage des données	3
3.2	Ré-échantillonnage horaire	4
3.3	Variables temporelles	4
3.4	Illustration du signal	4
4	Normalisation et création des séquences	5
4.1	Normalisation	5
4.2	Construction des fenêtres temporelles	5
4.3	Découpage train/test	5
5	Architecture du modèle	5
5.1	Choix d'un LSTM empilé	5
5.2	Structure interne	5
5.3	Pourquoi utiliser la dernière sortie temporelle?	6
5.4	Paramètres principaux	6
6	Stratégie d'entraînement	6
6.1	Fonction de coût	6
6.2	Optimisation	6
6.3	Boucle d'entraînement	7
6.4	Courbe de convergence	7
7	Prédictions et interprétation	7
7.1	Retour à l'échelle réelle	7
7.2	Comparaison observation / prédiction	8
8	Évaluation quantitative	8
8.1	Métriques utilisées	8
8.2	Résultats	9
9	Analyse critique	9

9.1	Points forts de la solution	9
9.2	Limites	9
9.3	Pistes d'amélioration	9
10	Conclusion	10
A	Correspondance avec le notebook	10

1 Contexte et objectif

La prévision de séries temporelles est un problème central en apprentissage profond, en particulier lorsque l'on souhaite anticiper une variable de consommation énergétique à partir d'un historique multivarié. Dans ce projet, l'objectif est de prédire la puissance active globale `Global_active_power` d'un foyer à partir de plusieurs variables électriques et temporelles, en exploitant un réseau **LSTM empilé à trois couches**.

Le problème est formulé comme une régression supervisée sur séquences. À partir d'une fenêtre temporelle d'observation, le modèle doit estimer la consommation à l'instant suivant. L'approche retenue s'appuie sur le jeu de données *Individual Household Electric Power Consumption* publié par l'UCI, qui contient des mesures à la minute sur plusieurs années.

2 Données utilisées

2.1 Source et nature du jeu de données

Le dataset provient de l'archive UCI et contient des mesures de consommation électrique domestique, collectées à une fréquence minute. Le fichier brut est volumineux, ce qui rend impossible une construction naïve de séquences directement sur la granularité initiale sans risquer une saturation mémoire.

Le notebook prend en charge deux modes d'accès aux données :

- téléchargement automatique depuis l'UCI si le fichier n'est pas présent localement ;
- chargement manuel si l'archive n'a pas pu être récupérée.

2.2 Variables d'entrée

Après nettoyage, les variables numériques exploitées sont :

- `Global_active_power` : cible à prédire ;
- `Global_reactive_power` ;
- `Voltage` ;
- `Global_intensity` ;
- `Sub_metering_1`, `Sub_metering_2`, `Sub_metering_3` ;
- variables temporelles dérivées : `hour`, `dayofweek`, `is_weekend` ;
- encodage cyclique de l'heure : `hour_sin`, `hour_cos`.

Le jeu de variables finales contient 10 colonnes, dont la première correspond à la cible.

3 Prétraitement et ingénierie des variables

3.1 Nettoyage des données

Le fichier brut est lu avec le séparateur ; et les valeurs manquantes notées ?. Les colonnes `Date` et `Time` sont fusionnées pour construire un index temporel unique `datetime`. Les lignes contenant des valeurs manquantes sont supprimées, puis les données sont triées chronologiquement.

Cette étape est essentielle car les modèles séquentiels supposent une chronologie cohérente. Toute rupture dans l'ordre temporel fausserait la construction des fenêtres d'entrée.

3.2 Ré-échantillonnage horaire

Le dataset d'origine est mesuré à la minute. Or, la construction de séquences longues sur plusieurs millions de lignes aurait un coût mémoire et computationnel trop important pour une exécution raisonnable dans un environnement courant. Le notebook effectue donc un **ré-échantillonnage à l'heure** en calculant la moyenne des mesures sur chaque heure.

Ce choix apporte plusieurs avantages :

- réduction drastique du volume de données ;
- meilleure stabilité des séries pour une tâche de prévision à court terme ;
- simplification de l'interprétation des prédictions ;
- construction plus efficace des séquences d'entrée pour le LSTM.

3.3 Variables temporelles

La consommation électrique présente souvent des périodicités journalières et hebdomadaires. Pour aider le modèle à exploiter ces régularités, le notebook ajoute plusieurs attributs temporels :

- `hour` : heure de la journée ;
- `dayofweek` : jour de la semaine ;
- `is_weekend` : indicateur binaire pour les week-ends ;
- `hour_sin` et `hour_cos` : représentation cyclique de l'heure.

L'encodage cyclique est particulièrement pertinent car l'heure 23h et l'heure 0h sont proches dans la réalité, alors qu'un encodage brut les placerait artificiellement aux extrémités d'un intervalle numérique. La transformation sinus/cosinus conserve la continuité du cycle journalier.

3.4 Illustration du signal

La figure suivante représente un extrait de la série de consommation sur deux semaines. Elle permet de visualiser les variations quotidiennes et les cycles de charge.

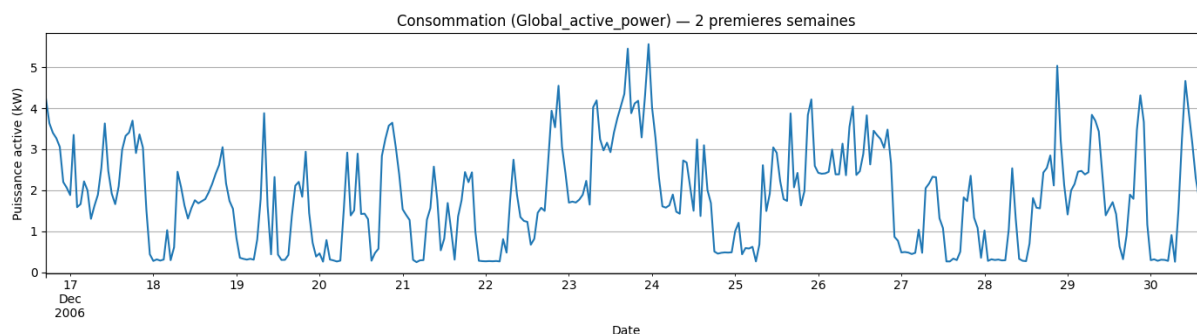


FIGURE 1 – Aperçu de la consommation active sur les deux premières semaines après ré-échantillonnage horaire.

4 Normalisation et création des séquences

4.1 Normalisation

Toutes les variables sont normalisées avec un `MinMaxScaler`. Une précaution importante est prise : le scaler est ajusté **uniquement sur la partie d'entraînement**, ce qui évite toute fuite d'information depuis le test vers l'apprentissage.

Un second scaler est utilisé pour la cible seule, afin de revenir ensuite à l'échelle réelle de la puissance active en kW lors de l'évaluation.

4.2 Construction des fenêtres temporelles

Le modèle est entraîné sur des fenêtres de longueur fixe. Dans le notebook, la longueur de séquence est de 24 pas temporels, soit 24 heures d'historique pour prédire la consommation de l'heure suivante.

Formellement, si l'on note $\mathbf{x}_t \in \mathbb{R}^d$ le vecteur des d variables à l'instant t , alors une séquence d'entrée est donnée par :

$$\mathbf{X}_t = [\mathbf{x}_{t-23}, \mathbf{x}_{t-22}, \dots, \mathbf{x}_t],$$

et la cible associée est :

$$y_t = x_{t+1}^{(0)},$$

où l'indice (0) désigne la première variable, c'est-à-dire `Global_active_power`.

4.3 Découpage train/test

Le découpage est réalisé de manière chronologique, sans mélange aléatoire, afin de respecter la structure temporelle du problème. Cette contrainte est indispensable en prévision de séries temporelles, car il serait incorrect d'entraîner le modèle sur des observations postérieures aux données utilisées pour le test.

Les séquences sont ensuite injectées dans des `DataLoader` PyTorch. Le lot d'apprentissage est mélangé à l'intérieur de l'ensemble d'entraînement, mais la séparation temporelle globale est conservée.

5 Architecture du modèle

5.1 Choix d'un LSTM empilé

Le cœur du projet est un réseau **Stacked LSTM** à 3 couches. Un LSTM est adapté à l'apprentissage de dépendances temporelles, car sa mémoire interne permet de conserver des informations utiles sur plusieurs pas de temps. L'empilement de plusieurs couches augmente la capacité du modèle à apprendre des représentations hiérarchiques de la dynamique de consommation.

5.2 Structure interne

Le modèle implémenté dans le notebook comprend :

- une couche d'entrée LSTM recevant les 10 variables par pas de temps ;
- trois couches LSTM avec `hidden_size = 128` ;

- un taux de dropout de 0.3 entre les couches récurrentes ;
- une couche linéaire finale projetant le dernier état caché vers une unique sortie scalaire.

L'architecture peut être résumée par :

$$\text{Input} \rightarrow \text{LSTM}_1 \rightarrow \text{LSTM}_2 \rightarrow \text{LSTM}_3 \rightarrow \text{Linear} \rightarrow \hat{y}_{t+1}.$$

5.3 Pourquoi utiliser la dernière sortie temporelle ?

Pour une tâche de prédiction au pas suivant, la dernière sortie du LSTM résume l'information contenue dans la fenêtre d'observation. Le modèle prend donc la représentation de l'instant final de la séquence et la transforme via une couche dense pour produire la prédiction.

5.4 Paramètres principaux

Paramètre	Valeur
Longueur de séquence	24 heures
Nombre de variables	10
Taille cachée	128
Nombre de couches LSTM	3
Dropout	0.3
Fonction de perte	MSE
Optimiseur	Adam
Taux d'apprentissage	10^{-3}
Taille de lot	64
Nombre d'époques	30

TABLE 1 – Hyperparamètres utilisés dans le notebook.

6 Stratégie d'entraînement

6.1 Fonction de coût

Le réseau est optimisé avec la **Mean Squared Error** (MSE) :

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2.$$

Cette fonction pénalise davantage les erreurs importantes, ce qui est pertinent lorsque l'on souhaite éviter des écarts trop élevés sur les pics de consommation.

6.2 Optimisation

L'optimiseur Adam est utilisé pour son efficacité pratique sur les problèmes de régression avec réseaux profonds. Il combine un pas d'apprentissage adaptatif et des estimations de moments du gradient, ce qui stabilise souvent l'entraînement des architectures récurrentes.

6.3 Boucle d'entraînement

Le notebook définit une fonction d'époque qui prend en charge deux modes : apprentissage et évaluation. En mode apprentissage, les gradients sont calculés, rétro-propagés, puis appliqués à l'aide de `optimizer.step()`. En mode évaluation, le calcul des gradients est désactivé afin de réduire le coût mémoire et d'éviter toute mise à jour des poids.

Les pertes d'entraînement et de test sont suivies à chaque époque. L'affichage est effectué périodiquement, tous les 5 epochs environ, pour surveiller la convergence.

6.4 Courbe de convergence

La courbe suivante doit être générée à partir des résultats du notebook une fois l'entraînement exécuté.

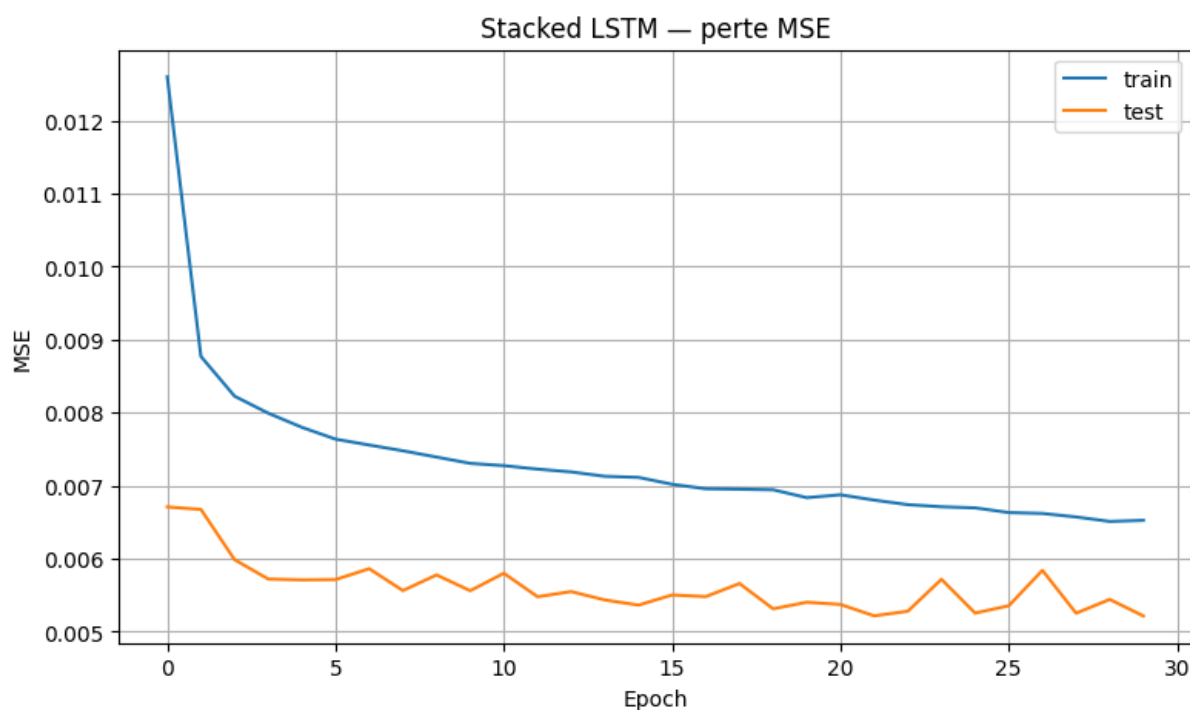


FIGURE 2 – Évolution de la perte MSE sur l'ensemble d'entraînement et l'ensemble de test.

7 Prédictions et interprétation

7.1 Retour à l'échelle réelle

Les prédictions du réseau sont d'abord obtenues dans l'espace normalisé, puis reconverties en kW via le scaler dédié à la cible. Cette étape est indispensable pour interpréter les résultats dans l'unité physique d'origine.

7.2 Comparaison observation / prédiction

Le notebook compare ensuite, sur les 500 premiers points de test, la série réelle et la série prédite. Cette visualisation permet d'apprécier :

- la capacité du modèle à suivre la tendance générale ;
- la précision sur les oscillations rapides ;
- les éventuels décalages temporels ou lissage excessif ;
- la qualité de la reconstruction des pics de consommation.

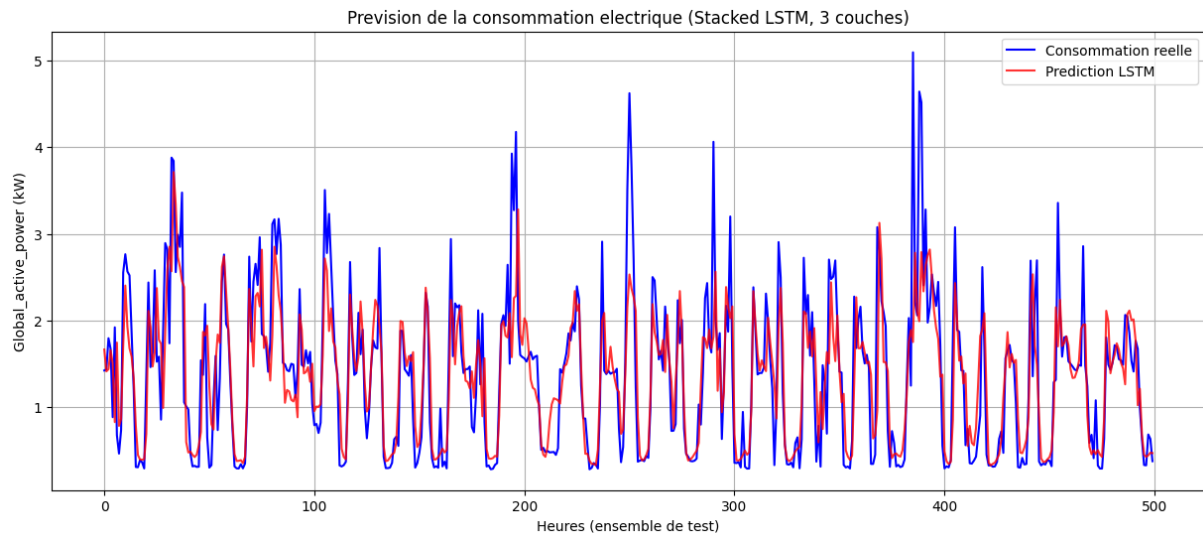


FIGURE 3 – Comparaison entre la consommation réelle et la prédiction du Stacked LSTM sur une portion du test.

8 Évaluation quantitative

8.1 Métriques utilisées

Plusieurs métriques sont calculées pour évaluer la qualité du modèle :

- **MAE** : erreur absolue moyenne ;
- **RMSE** : racine de l'erreur quadratique moyenne ;
- R^2 : coefficient de détermination ;
- **MAPE** : erreur absolue moyenne en pourcentage.

Leur définition mathématique est la suivante :

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|,$$

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2},$$

$$R^2 = 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2},$$

$$\text{MAPE} = \frac{100}{N} \sum_{i=1}^N \left| \frac{y_i - \hat{y}_i}{\max(|y_i|, \varepsilon)} \right|,$$

où ε évite la division par zéro.

8.2 Résultats

Le notebook exporte les métriques dans un fichier CSV situé dans `figs/tp2_results.csv`. Si le notebook n'a pas encore été exécuté, cette table peut être renseignée après entraînement.

Métrique	Valeur	Unité
MAE	<i>à compléter</i>	kW
RMSE	<i>à compléter</i>	kW
R^2	<i>à compléter</i>	—
MAPE	<i>à compléter</i>	%

TABLE 2 – Tableau de synthèse des performances sur l'ensemble de test.

9 Analyse critique

9.1 Points forts de la solution

Plusieurs choix méthodologiques renforcent la robustesse du projet :

- le ré-échantillonnage horaire limite la complexité mémoire ;
- le découpage chronologique respecte la nature temporelle du problème ;
- l'encodage cyclique de l'heure introduit une information périodique pertinente ;
- la normalisation sur l'entraînement uniquement évite la fuite d'information ;
- la structure Stacked LSTM permet d'apprendre des dépendances séquentielles plus riches qu'un modèle linéaire.

9.2 Limites

Le projet reste néanmoins perfectible :

- la moyenne horaire peut lisser les pics de consommation importants ;
- le modèle ne prend pas en compte de variables exogènes comme la météo ou les jours fériés ;
- aucune stratégie d'arrêt anticipé n'est décrite ;
- aucun réglage fin systématique des hyperparamètres n'est présenté ;
- une architecture plus avancée pourrait mieux capturer certaines non-linéarités.

9.3 Pistes d'amélioration

Des extensions naturelles du projet sont possibles :

- comparaison avec un GRU, un TCN ou un Transformer temporel ;
- ajout de variables calendaires plus fines ;

- apprentissage multi-horizon au lieu d’une seule heure future ;
- utilisation d’un **EarlyStopping** et d’une validation dédiée ;
- recherche d’hyperparamètres automatisée ;
- retour à la granularité minute avec une stratégie mémoire plus efficace.

10 Conclusion

Ce projet illustre une chaîne complète de prévision de série temporelle multivariée : acquisition des données, nettoyage, agrégation horaire, enrichissement temporel, normalisation, construction de séquences, entraînement d’un réseau LSTM empilé et évaluation sur données réelles.

L’approche retenue est cohérente avec les contraintes du problème et avec les ressources disponibles. Le Stacked LSTM constitue une base solide pour modéliser la dynamique de consommation électrique, en particulier lorsqu’il est combiné à des variables temporelles bien choisies et à une séparation chronologique stricte entre apprentissage et test.

A Correspondance avec le notebook

Pour faciliter la lecture du code, les étapes du rapport correspondent aux blocs du notebook :

1. imports et configuration ;
2. chargement des données ;
3. ré-échantillonnage horaire et feature engineering ;
4. normalisation et création des séquences ;
5. définition du modèle Stacked LSTM ;
6. entraînement ;
7. prédictions et retour à l’échelle réelle ;
8. évaluation finale.